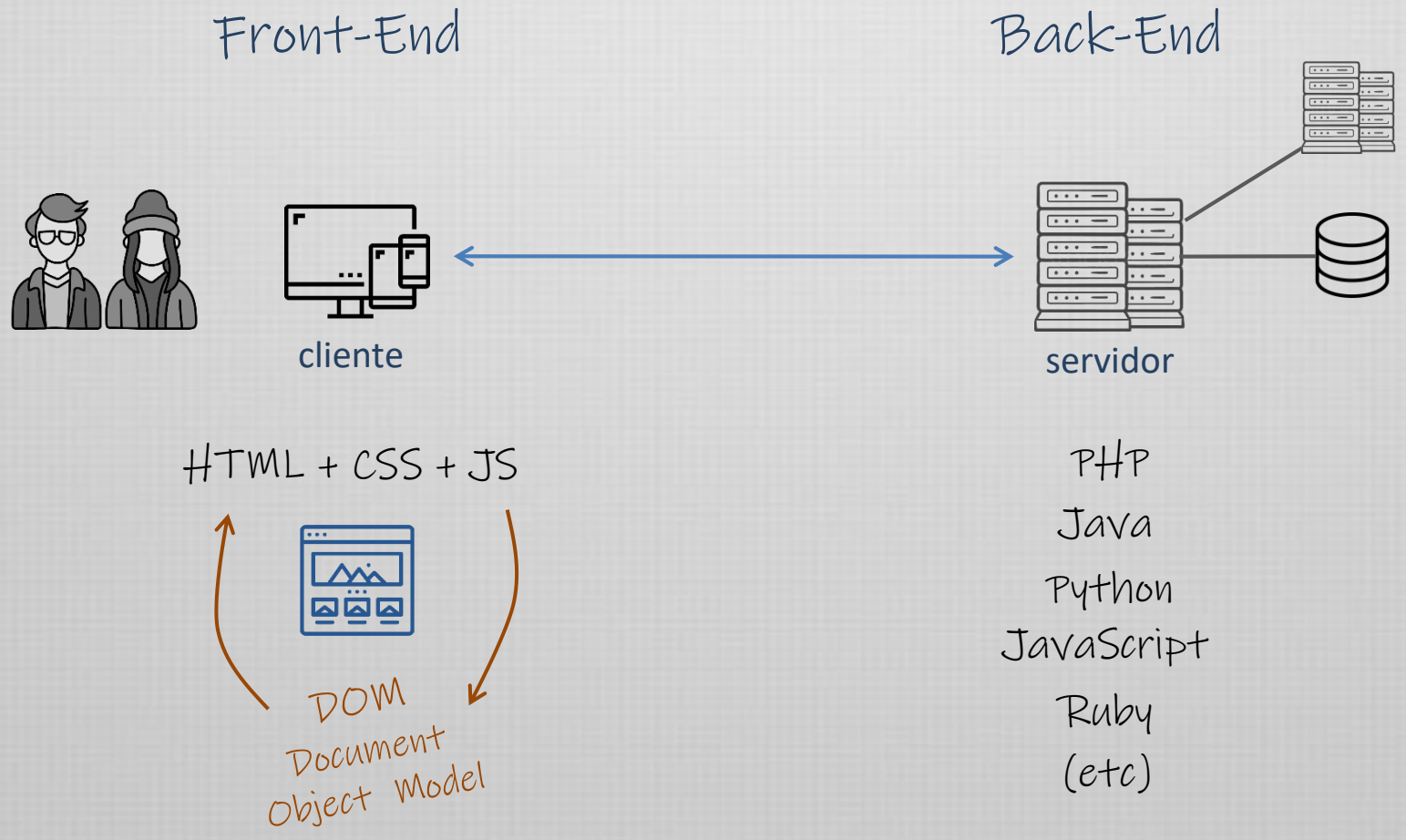
Ingeniería de Aplicaciones Web

Diego C. Martínez

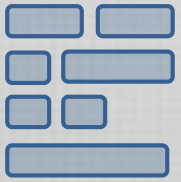
Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur

Escenario web



Escenario web

```
...
<div>...</div>
<div>
 <Contacto>
</div>
<p>
...
```



```
function Contacto() {

    return (
        <div>…</div>
     );
}

export default Contacto;
```

Escenario web

GET + URL A	---------->	Realizar acción X
GET + URL B	---------->	Realizar acción Y
POST + URL C	---------->	Realizar acción Z

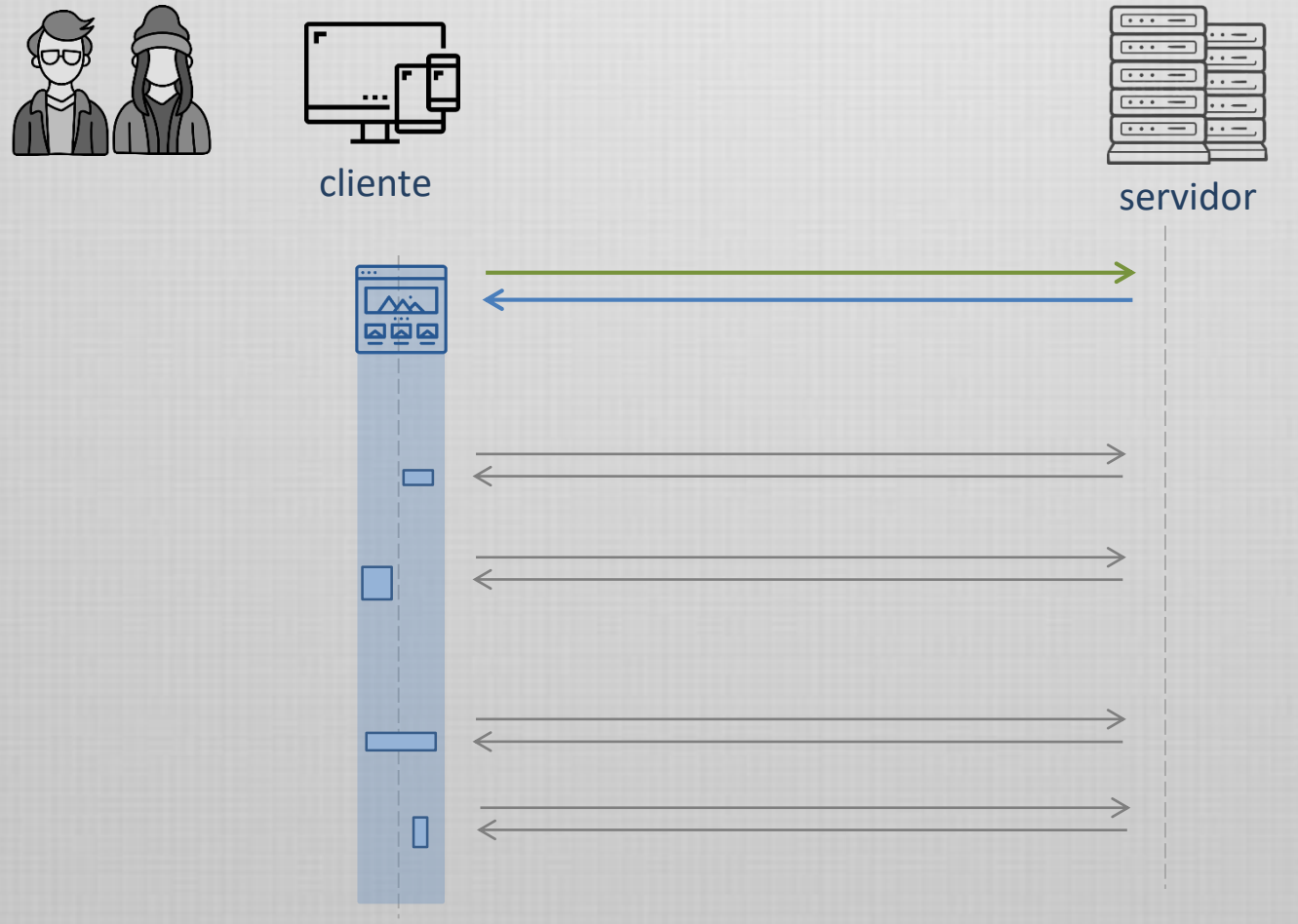
La acción puede ser devolver una página, o un recurso (png, jpg, json, o cualquier acción de procesamiento general

Escenario web

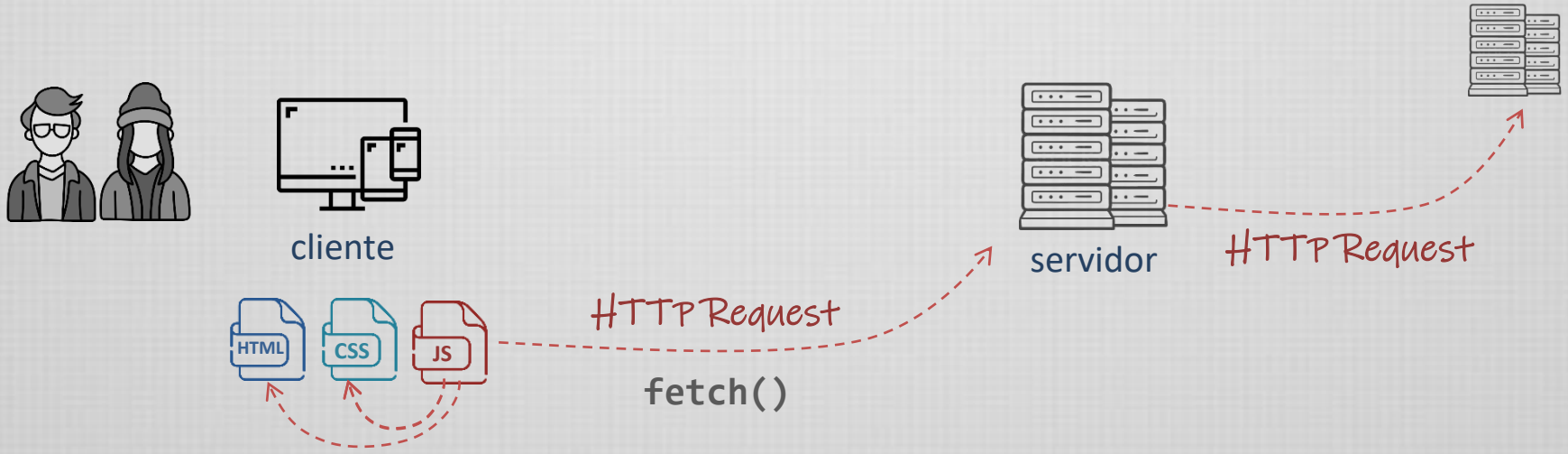
cliente

servidor

Escenario web



Escenario web



Escenario web

cliente

servidor

¿clima actual?

```
{
  estado: "lluvia",
  temp: 25;
  vientos:12;
  ...
}
```

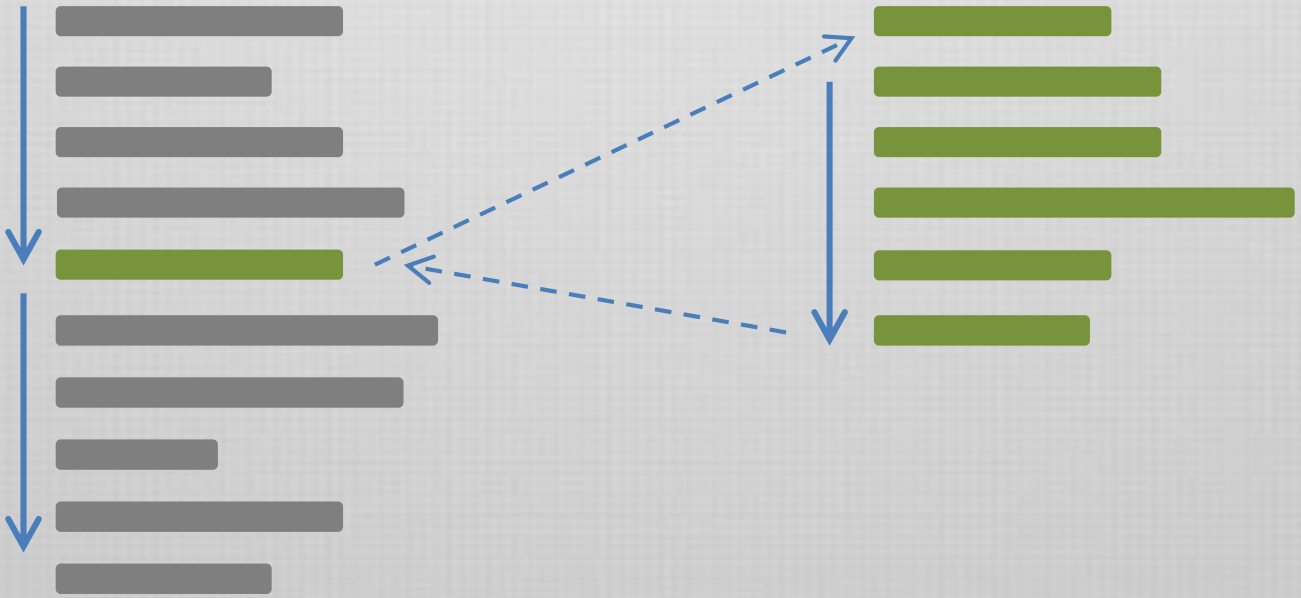
Es un pedido a un sistema externo

Todos los lenguajes para la web tienen mecanismos para esto

A nosotros nos interesa aquí JavaScript

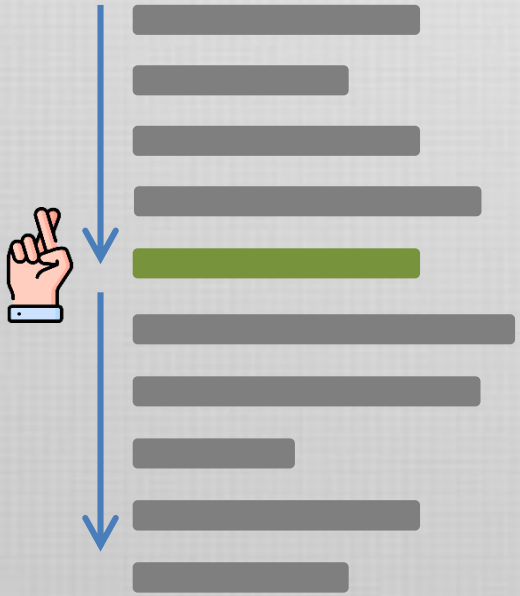
JavaScript es single-threaded

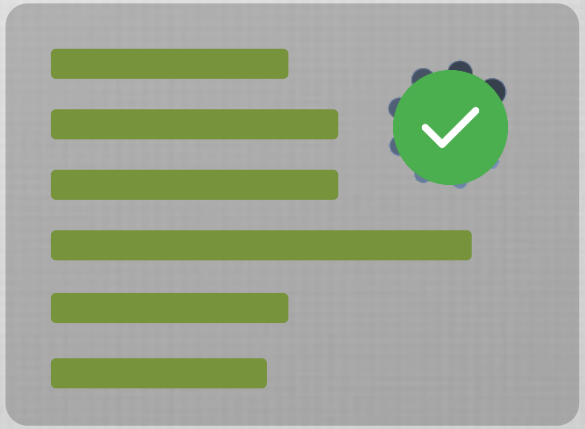
hace una tarea a la vez



JavaScript es single-threaded

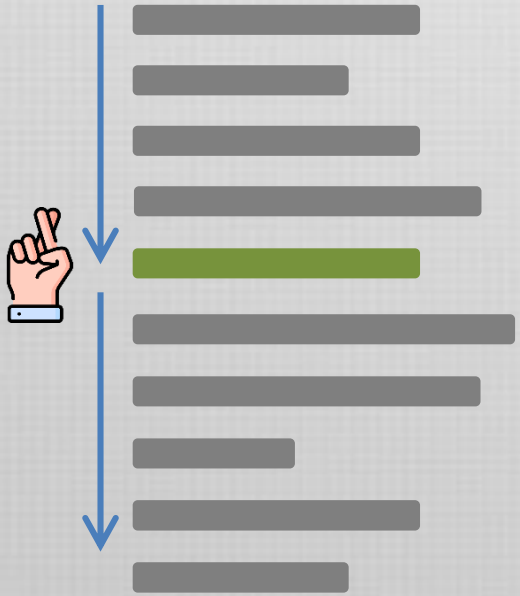
hace una tarea a la vez

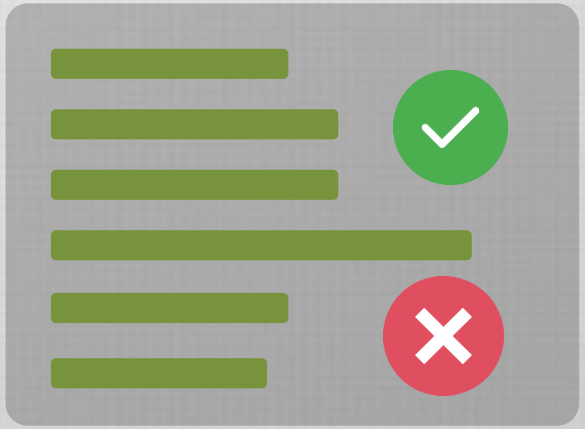




JavaScript es single-threaded

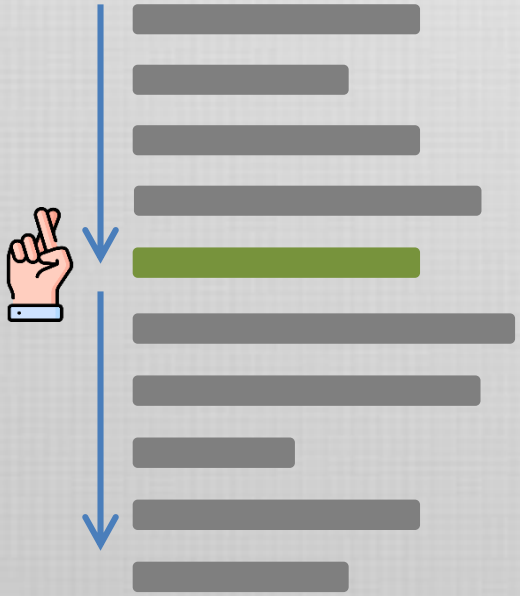
hace una tarea a la vez

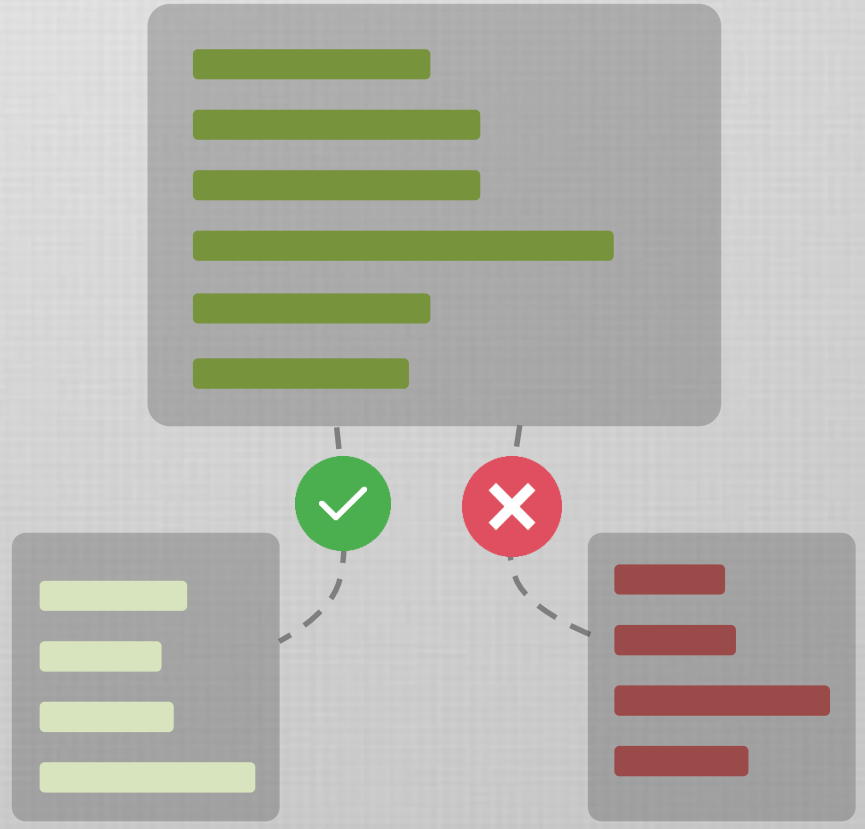




JavaScript es single-threaded

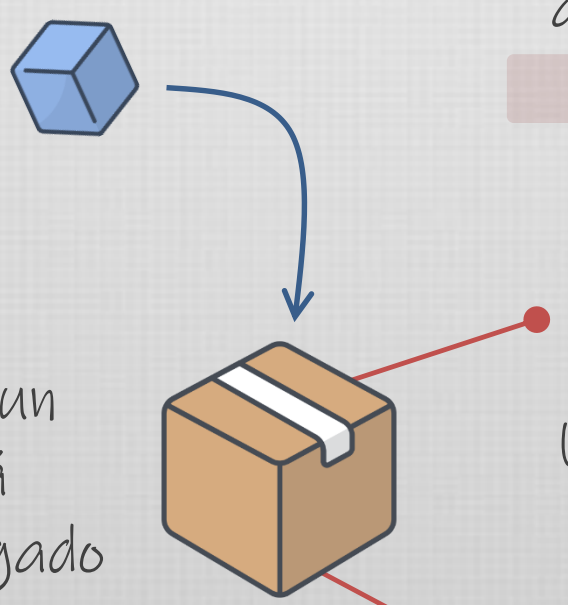
hace una tarea a la vez





Promesas

Las operaciones asincrónicas como `fetch()` devuelven promesas



Un contenedor para un resultado que será eventualmente entregado

`then()`

Un objeto en el que podemos registrar oyentes

`catch()`

promesa tambien!

```
unaPromesa.then(...).catch(...)
```

```
accion1;
accion2;
...
```

```
(dato) => {...}
```

(lo más habitual)

Promesas

```
tareaAsincrona()
.then(...)
.then(...)
.then(...)
.catch(...)
```

Cada **then** recibe
el dato del anterior

```
fetch("...")
.then(r => r.json())
.then(data => {return data.length;})
.then(len => console.log(len));
```

¿Que pasa si no devuelvo nada en un paso intermedio?

¿Que pasa si devuelvo otra promesa en un paso intermedio?

Si bien por lo general consumiremos promesas, también podemos crearlas

```
const promesa = new Promise(
    function(resolve, reject){
        // codigo que produce algun resultado
    }
)
```

resolve()

reject()

El parámetro de las funciones es solo un dato referencial

resolve(10)

resolve("ok")

Escenario web

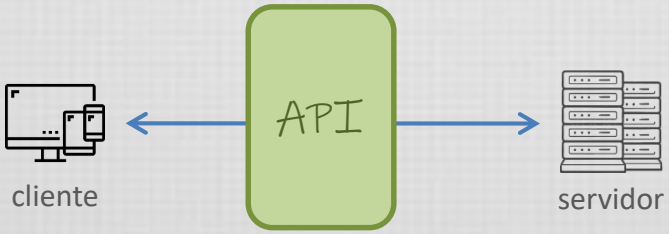
cliente

API

http://sitio.com/abc
http://sitio.com/xyz/1
http://sitio.com/11/22/33

Servicio Web
(Web Service)

APIs



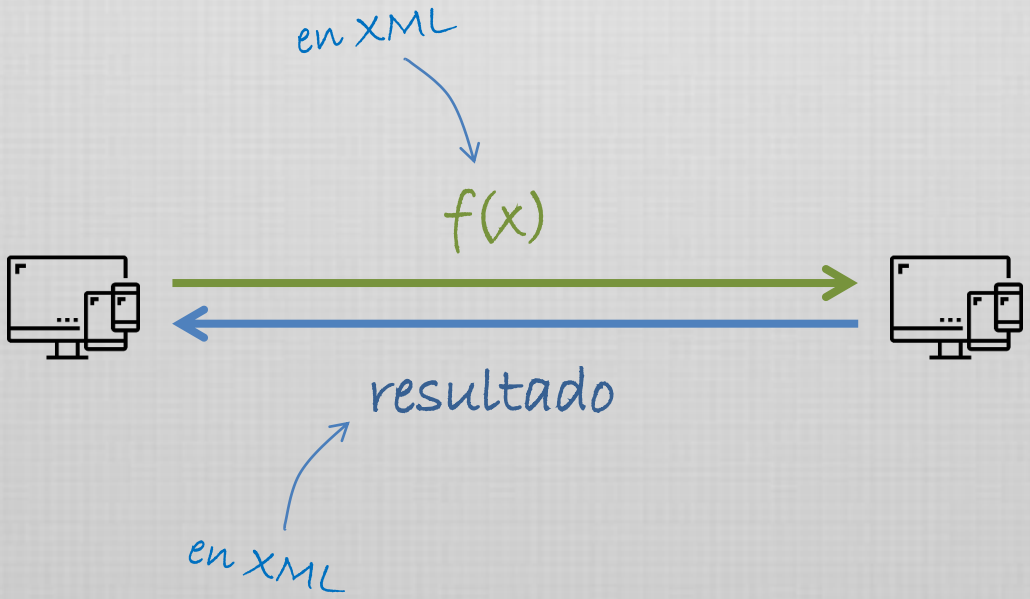
Un servicio web es una
aplicación accedida remotamente usando protocolos de Internet, y
que utiliza XML/JSON como mecanismo de mensajes

XML-RPC
JSON-RPC
SOAP
REST

Web Service – XML-RPC

XML-based Remote Procedure Call

Es simplemente la invocación remota de funciones via web



Web Service – XML-RPC

XML-based Remote Procedure Call

Es simplemente la invocación remota de funciones via web

Web Service – XML-RPC

XML-based Remote Procedure Call

Es simplemente la invocación remota de funciones via web

```
<?xml version="1.0"?>
<methodCall>
   <methodName>circleArea</methodName>
   <params>
     <param>
       <value><double>2.41</double></value>
     </param>
   </params>
</methodCall>
```

```
<?xml version="1.0"?>
<methodResponse>
  <params>
    <param>
      <value><double>
            18.246684
      </double></value>
    </param>
  </params>
</methodResponse>
```

Web Service – SOAP

SOAP es un mecanismo general para la interoperabilidad de sistemas

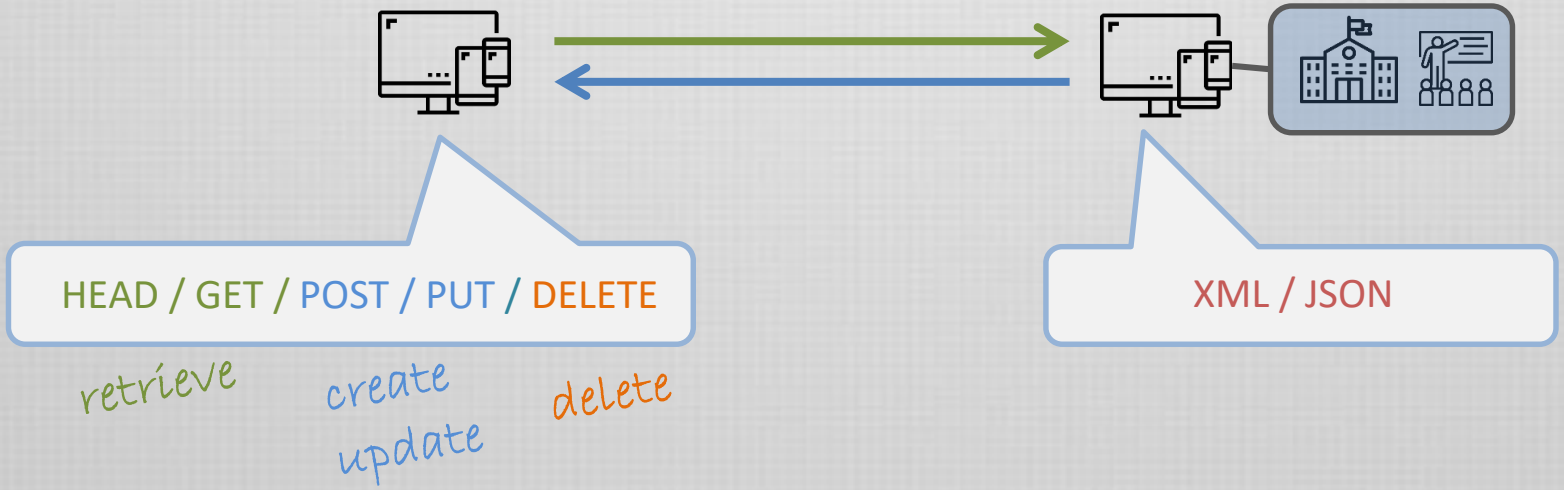
```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <SOAP-ENV:Body>
    <ns1:getTemp xmlns:ns1="urn:xmethods-Temperature"
       SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
       <zipcode xsi:type="xsd:string">10016</zipcode>
    </ns1:getTemp>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope
       xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:xsd="http://www.w3.org/2001/XMLSchema">
       <SOAP-ENV:Body>
           <ns1:getTempResponse xmlns:ns1="urn:xmethods-Temperature"
            SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
                <return xsi:type="xsd:float">71.0</return>
           </ns1:getTempResponse>
       </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Web Service – REST

REST es una metodología simple para la implementación de web services

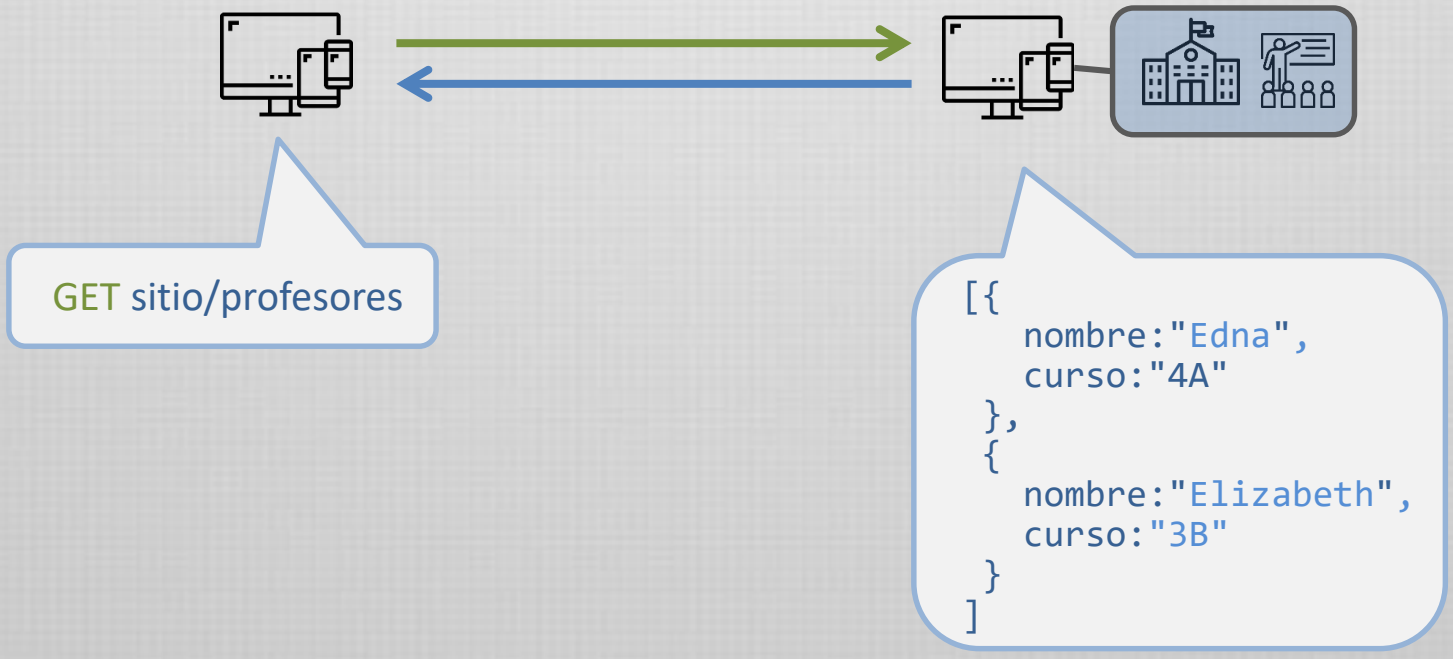
Es la forma más habitual en las interacciones de servicios web



Los verbos HTTP son las acciones del cliente REST

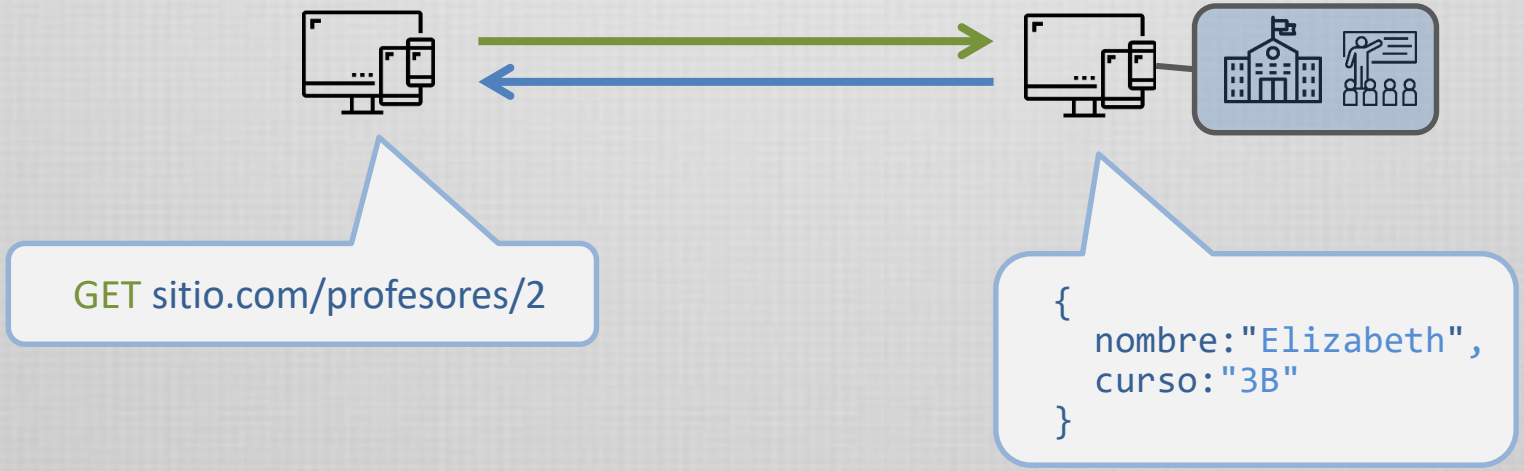
Web Service – REST

REST es una metodología simple para la implementación de web services



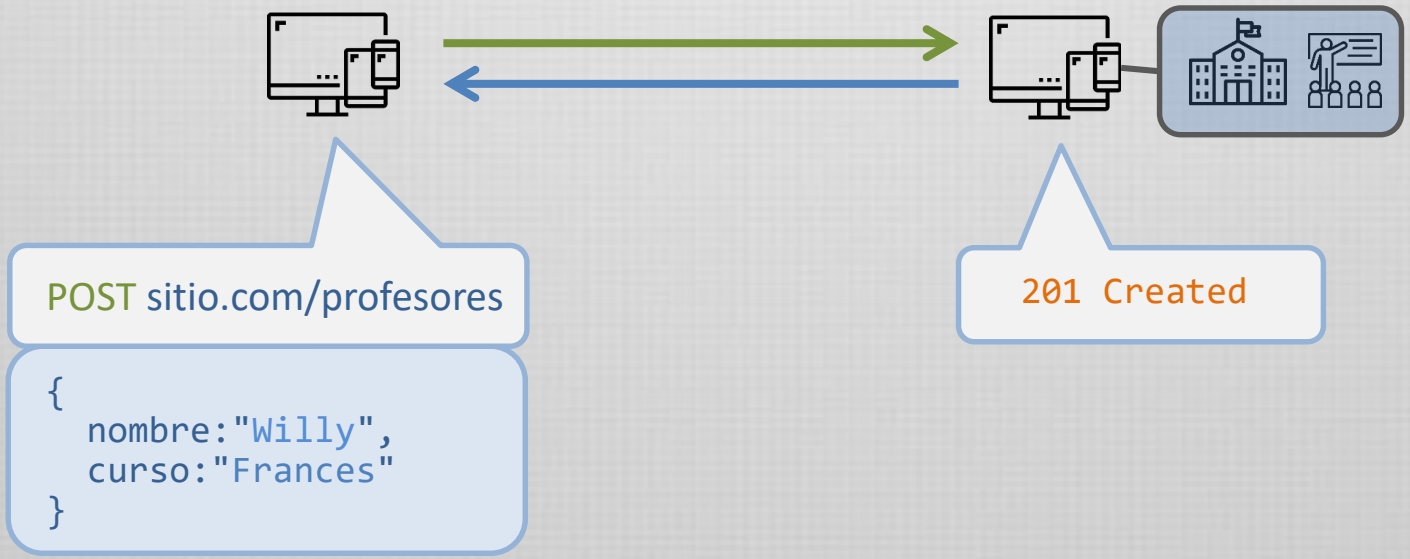
Web Service – REST

REST es una metodología simple para la implementación de web services



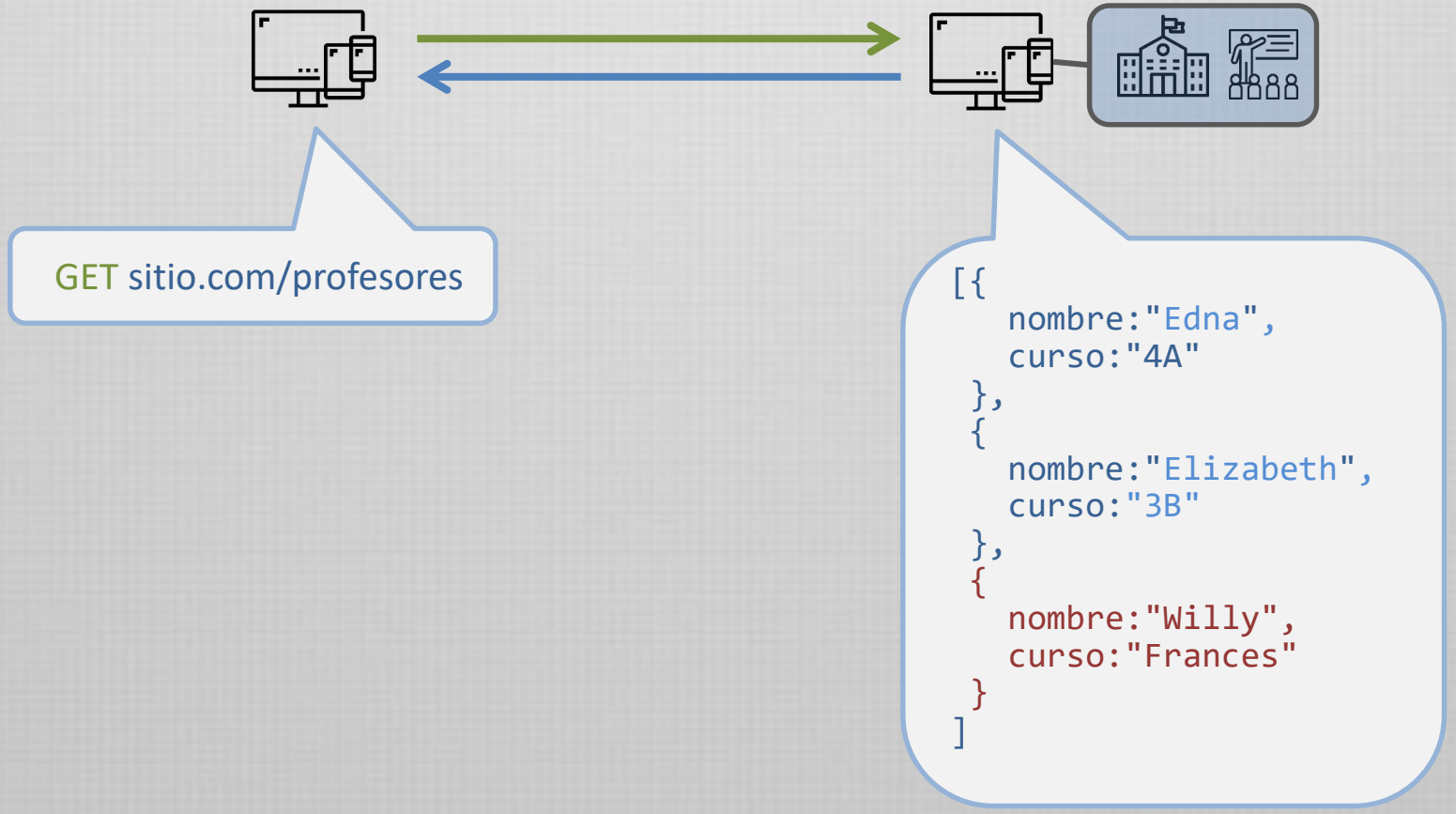
Web Service – REST

REST es una metodología simple para la implementación de web services



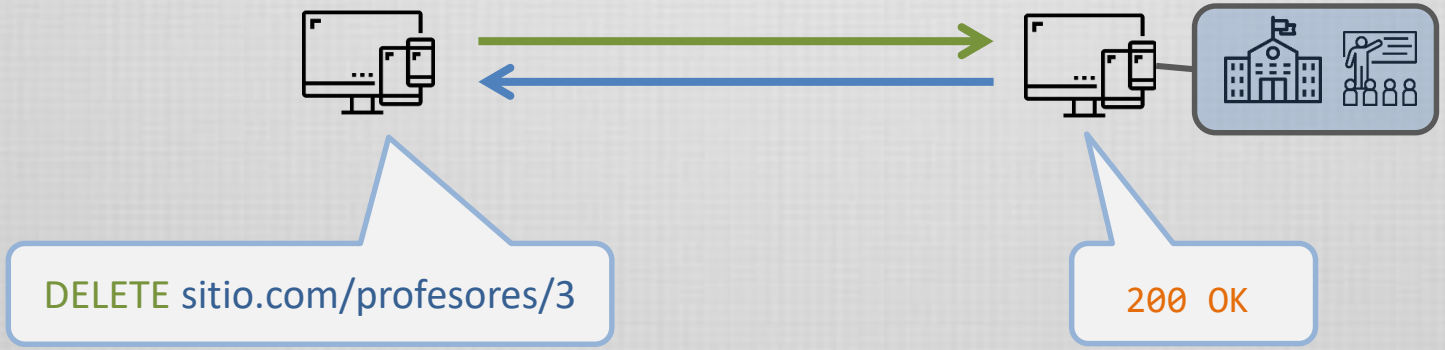
Web Service – REST

REST es una metodología simple para la implementación de web services



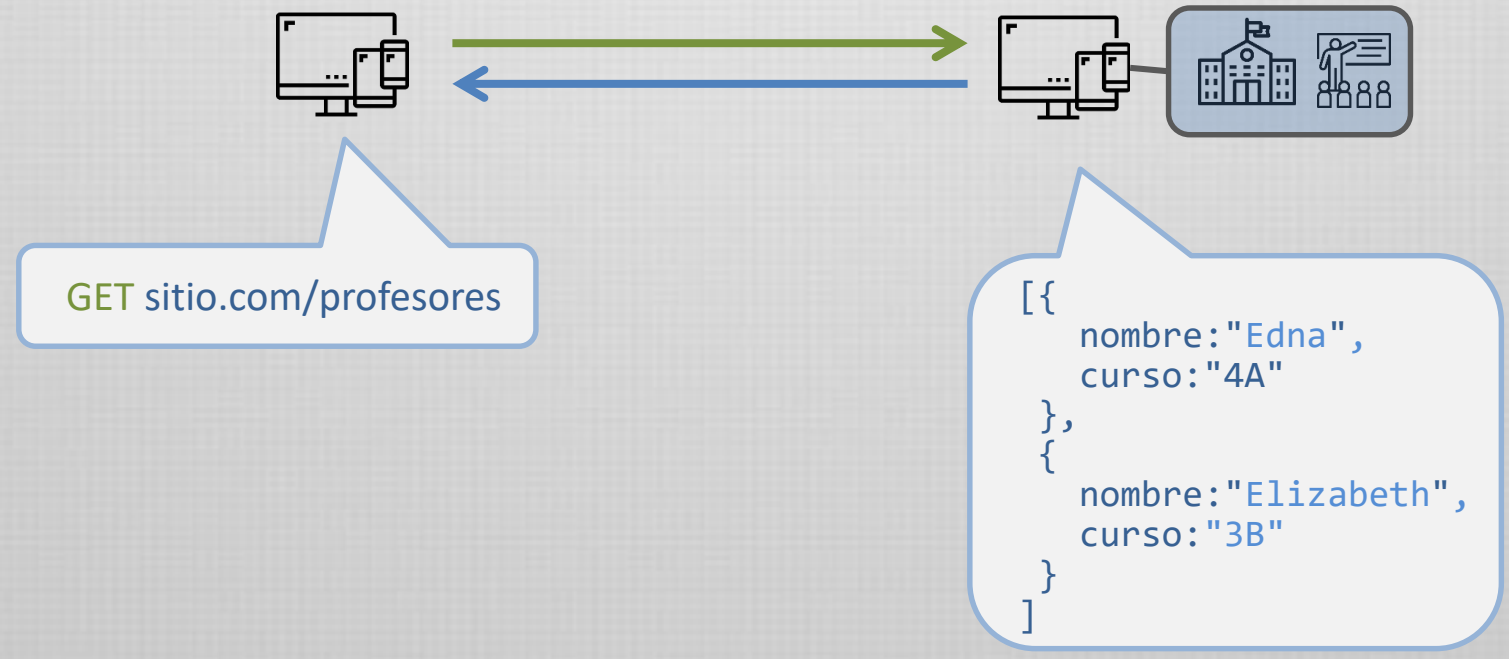
Web Service – REST

REST es una metodología simple para la implementación de web services



Web Service – REST

REST es una metodología simple para la implementación de web services



API / Path design

http://unsitio.com/abc/def/123

http://unsitio.com/abc/def/123/xy

http://unsitio.com/abc?param1=zz&param2=32

http://unsitio.com/ghi

http://unsitio.com/ghi/jkl

http://unsitio.com/ghi/jkl/mno